vLLM Omni Neuron Beta 1 (05/20/2026)

Overview

Please note: THIS IS BETA SOFTWARE. PLEASE SHARE YOUR FEEDBACK AND EXPERIENCE ONLY WITH NEURON TEAM FOR IMPROVEMENTS.

This guide walks through running the Wan 2.2 Text-to-Video (14B) model on AWS Trainium (trn2) using the vllm-omni-neuron private beta container image.

What's new in this beta release:

- First release of plugin for vLLM Omni for Trainium
- Functional support of Wan 2.2 Text-to-Video (14B) model on AWS Trainium (trn2)

Prerequisites

- Launch a Trn2 or Trn3 instance. Note the container and instructions below are tested on `trn2.48xlarge` instance (16 Neuron devices), however the container should work on any Trn2 or Trn3 instance
- Docker installed

Step 1 — Pull the Image

```
export IMAGE=421672808698.dkr.ecr.us-east-1.amazonaws.com/concourse-release-1cb0647:pr-655cd3ee9b8d69818e52f37fbe1bb2a445bfbd60

docker pull "$IMAGE"
```

Step 2 — Install the Neuron Driver

The container ships a matched Neuron driver RPM. Extract and install it on the host:

```
# Extract driver RPM from the container image
TMP=$(mktemp -d)
docker create --name extract-driver "$IMAGE"
docker cp extract-driver:/opt/aws/neuron/driver/. "$TMP/"
docker rm extract-driver

# Install on Ubuntu/Debian host
sudo apt-get install -y alien
cd "$TMP" && sudo alien --to-deb --scripts aws-neuronx-dkms-*.noarch.rpm
sudo dpkg -i aws-neuronx-dkms_*.deb

# Verify
modinfo neuron | grep "^version"
```

For RPM-based hosts (Amazon Linux, RHEL):

```
sudo rpm -i "$TMP"/aws-neuronx-dkms-*.noarch.rpm
```

Step 3 — Start the Container

```
sudo docker run -d --privileged \
  --name wan22 \
  --shm-size=2g \
  -v /path/to/hf_cache:/root/.cache/huggingface \
  -v /tmp/vllm_neff_cache:/root/.cache/vllm \
  -v /tmp/vllm_cache:/tmp/vllm_cache \
  -v /tmp/nki_cache:/tmp/nki_cache \
  --device /dev/neuron0 --device /dev/neuron1 --device /dev/neuron2 --device /dev/neuron3 \
  --device /dev/neuron4 --device /dev/neuron5 --device /dev/neuron6 --device /dev/neuron7 \
  --device /dev/neuron8 --device /dev/neuron9 --device /dev/neuron10 --device /dev/neuron11 \
  --device /dev/neuron12 --device /dev/neuron13 --device /dev/neuron14 --device /dev/neuron15 \
  "$IMAGE" \
  sleep infinity
```

Notes:

- `-v /path/to/hf_cache:/root/.cache/huggingface` caches the model weights (~116 GB) so subsequent runs skip the download.
- `-v /tmp/vllm_neff_cache:/root/.cache/vllm` persists the compiled Neuron NEFFs (DiT + VAE graphs). with this mount, subsequent runs reuse them and skip recompilation.
- `--shm-size=2g` is required. Rank 0 returns the decoded video to the engine through a POSIX shared-memory segment in `/dev/shm`. The full-mode output does not fit in Docker's 64 MB default `/dev/shm`, and the resulting page fault causes a silent SIGBUS on rank 0. 2 GB matches vLLM's documented recommendation and leaves ample headroom.
- All 16 Neuron devices must be passed explicitly with `--device`.

IDENTIFYING NEFF CACHE ENTRIES

Each compiled NEFF is stored under the cache directory (`/root/.cache/vllm/<hash>/` inside the container, or `/tmp/vllm_neff_cache/<hash>/` on the host with the volume mount from Step 3). To identify which model component a NEFF corresponds to, inspect `example_inputs.txt`:

```
# Inside the container:
for dir in /root/.cache/vllm/*/; do
  key=$(basename "$dir")
  shape=$(grep -A1 "Input 0:" "$dir/example_inputs.txt" | grep "Shape:" | head -1 | sed
's/.*Shape: //')
  size=$(du -sh "$dir/graph.neff" | cut -f1)
  echo "$key | $size | input0=$shape"
done
```

`Input 0` is the latent tensor with shape `(batch, z_dim, T_lat, H_lat, W_lat)`:

- `z_dim = 16` — Wan2.2 VAE latent channel count
- `H_lat = ((height // 16) * 16) / 8`, `W_lat = ((width // 16) * 16) / 8` — pipeline rounds H/W down to a multiple of 16 (= `vae_scale_factor_spatial × patch_size`), then divides by 8
- `T_lat = (num_frames - 1) / 4 + 1` — VAE temporal scale factor is 4

Component mapping (validated against the cache for both modes):

	Input 0 shape	Component	Dev (120×208, 5 frames)	Full (480×832, 81 frames)
1	`(1, 512)` int64 (token ids)	Text encoder (UMT5)	`(1, 512)`	`(1, 512)` (shared with dev)
2	`(1, 16, 1, H_lat, W_lat)` bf16 (single latent frame)	VAE decoder	`(1, 16, 1, 14, 26)`	`(1, 16, 1, 60, 104)`
3	`(1, 16, T_lat, H_lat, W_lat)` bf16 with `T_lat ≥ 2`	DiT transformer	`(1, 16, 2, 14, 26)`	`(1, 16, 21, 60, 104)`

For one model at one resolution, expect **11 NEFFs: 8 TE + 2 VAE + 1 DiT**.

Step 4 — Run the Example

DEV MODE (QUICK TEST — 5 FRAMES, LOW RESOLUTION, 3 STEPS)

```
sudo docker exec wan22 python /workspace/plugin/examples/wan22/run.py --dev
```

FULL MODE (81 FRAMES, 480×832, 40 STEPS)

```
sudo docker exec wan22 python /workspace/plugin/examples/wan22/run.py
```

Available flags:
Note that example script mentioned above loads vLLM-Omni configurations from wan22_stage.yaml

	Flag	Default	Description
1	`--dev`	off	Dev mode: 5 frames, 120×208, 3 steps
2	`--tensor-parallel-size`	8	Number of Neuron devices for tensor parallelism
3	`--height`	480	Video height
4	`--width`	832	Video width
5	`--num-frames`	81	Number of frames to generate
6	`--prompt`	(cat example)	Text prompt
7	`--output`	`wan_output.mp4`	Output file path
8	`--model-path`	`Wan-AI/Wan2.2-T2V-A14B-Diffusers`	Local path or HuggingFace model ID
9	`--cache-backend`	None	Set to `cache_dit` for acceleration
10	`--no-cfg`	off	Disable classifier-free guidance
11	`--guidance-scale`	5	CFG guidance scale

Step 5 - Review the Source code

If you would like to review the source code of the plugin and WAN 2.2 model implementation, you can find the source code in the container at the following location

```
/workspace/plugin/vllm_omni_neuron
```

Troubleshooting

`NRT_FAILURE IN NRT_INIT()` / `LOGICAL NEURON CORE(S) NOT AVAILABLE`

The host Neuron driver doesn't match the container's runtime. Install the driver baked into the image (Step 1).

COMPILER SCRATCH DIRECTORIES ACCUMULATING IN CWD

After running, you may see many short hex-named directories alongside `log-neuron-cc.txt` and `global_metric_store.json` in the container's working directory:

```
/workspace/
├── 12db1dcdf9b8269b/
│   └── neuronxcc.private_nkl.transpose.tiled_dve_transpose_10_lnc0_*.json
├── 3762b8b28236e9a0/
├── ...
├── log-neuron-cc.txt
├── global_metric_store.json
├── plugin/
└── wan_output.mp4
```

These are NKI kernel scratch directories written by `neuronx-cc` directly into `os.getcwd()`. They are not the NEFF cache (which lives under `/root/.cache/vllm/<32-char-hash>/`) and are safe to delete.

Known Limitations

- This Beta currently supports only functional implementation of WAN 2.2-14B model. More performant implementation is currently in progress and would be shared in future Betas.

Feedback and Support

Please report issues through your AWS Account team during this private beta.

Beta feedback survey: https://pulse.amazon/survey/4Q1WP4HW